star map generator notes, mostly so future me remembers why the code is shaped like this.

1 theory

the whole project is really just answering one question: given a star's fixed position in the sky, and my location + the current time, what part of the sky is it in *right now*?

so there's two coordinate systems fighting each other here.

equatorial coordinates (RA/dec) — this is how the catalog stores stars, and it's basically lat/long but for the sky. right ascension (RA) is like longitude, except measured in hours instead of degrees (0h to 24h), because old astronomers tied it to earth's 24hr rotation instead of just using 360° like normal people. declination (dec) is like latitude, $-90°$ to $+90°$. since stars are so far away, these barely change over a human lifetime — so one RA/dec pair per star works forever, which is why a catalog can just store one number and be done.

since 24h = 360°,

$$1^{\mathrm{h}} = 15° \tag{1}$$

so converting RA from hours to degrees is just multiply by 15.

horizontal coordinates (alt/az) — this is what we actually want to plot. altitude = how high above the horizon (0° = horizon, 90° = straight up, negative = below horizon = not visible). azimuth = compass direction, 0° north, 90° east, etc.

this is observer-dependent and time-dependent, which is the annoying part — the same star has a totally different alt/az depending on where you're standing and what time it is, because earth is spinning underneath it.

so the conversion RA/dec → alt/az needs: the star's RA/dec, my lat/long, and the exact time (to know the local sidereal time, i.e. what part of the sky is currently facing me). roughly it goes through an hour angle:

$$H = \mathrm{LST} - \mathrm{RA} \tag{2}$$

and then some spherical trig i am absolutely not doing by hand:

$$\sin(\mathrm{alt}) = \sin\delta\sin\phi + \cos\delta\cos\phi\cos H \tag{3}$$

squishing the dome flat - the result of these is a hemispehere, but a plot is flat, so

$$\theta = \mathrm{Az}, \qquad r = 90 - \mathrm{alt} \tag{4}$$

brightness — (Vmag) lower number = brighter. `MAG_LIMIT` is just "how faint will the plot plot."

color — comes from spectral class, which is really just surface temperature:

$$\mathrm{O} > \mathrm{B} > \mathrm{A} > \mathrm{F} > \mathrm{G} > \mathrm{K} > \mathrm{M} \tag{5}$$

hot = blue, cool = orange/red (sun is G, kind of yellow-white). only using the first letter of the spectral class string is a simplification but looks convincing enough.

2 code logic

alors j'essaie d'expliquer

parsing RA — the catalog gives RA as a string like `"00h 05m 09.9s"`, not a number, so

- strip the unit letters
- split on whitespace
- convert hours/min/sec into decimal hours

parsing Dec — same thing but degrees/arcmin/arcsec, and can be -ve, then

- grab the sign before stripping it off
- reapply sign after `float()`